FP-Smol

A Finite-Volume Fokker–Planck Solver for
Cloud Droplet Size Distributions

A Guide to the Physics, Numerics, and Python Codebase

Prepared as project documentation for the Python port of the MATLAB steady and time-dependent Fokker–Planck solvers developed for convection–cloud chamber microphysics research (ACDC2 / Genesis collaboration).

Contents

- 1. Why This Code Exists: The Physical Problem
- 2. The Governing Equation
- 3. Kohler Theory: Why Small Droplets Behave Differently
- 4. The Numerical Method: Finite Volumes + Scharfetter-Gummel
- 5. From Steady State to Time-Dependence
- 6. Codebase Tour
- 7. Installing and Running the Code
- 8. Reproducing the Benchmark Results
- 9. Validating the Time-Dependent Solver
- 10. Extending the Code: What Comes Next
- Appendix A: Symbol Reference
- Appendix B: Troubleshooting Common Issues

1. Why This Code Exists: The Physical Problem

Clouds form when water vapor condenses onto small airborne particles called aerosols. Once a droplet exists, it can keep growing in two very different ways: **condensation**, where water vapor diffuses onto the droplet's surface, and **collision-coalescence**, where droplets of different sizes fall at different speeds, collide, and merge. Real rain only happens once droplets are big enough for the second process to take over — a transition atmospheric scientists still cannot predict precisely.

This codebase focuses on the first stage: how a population of droplets grows by condensation inside a turbulent **convection–cloud chamber** (a laboratory device, such as Michigan Tech's "Pi Chamber," used to study these processes under controlled conditions). Rather than tracking every individual droplet, we track the evolution of the **droplet size distribution (DSD)** — essentially a histogram of how many droplets exist at each size, and how that histogram evolves as droplets grow, are removed by settling, and new ones are injected.

Three physical effects compete to shape this distribution:

- **Drift** — the deterministic tendency of droplets to grow (or shrink) due to the mean supersaturation in the chamber.
- **Diffusion** — random broadening of the size distribution caused by turbulent fluctuations in local supersaturation (some droplets briefly see more moisture than others).
- **Settling (sedimentation)** — larger droplets fall faster and leave the chamber, removing them from the population.

The interplay of these three effects is exactly what a **Fokker–Planck equation** describes, which is why that is the equation this code solves.

2. The Governing Equation

Rather than working directly with droplet radius *r*, the code works in the variable **x = r^2** (radius squared). This is a standard trick in cloud physics: condensation growth makes d(r^2)/dt roughly constant, so switching to x turns an awkward 1/r growth law into a much simpler constant-drift problem.

At **steady state** (the distribution no longer changes in time), the number density N(x) obeys a flux-conservation law:

$$dJ/dx + \lambda(x)\, N(x) = 0$$

where the flux J combines drift and diffusion,

$$J(x) = A_x(x)\, N(x) - D_x\, dN/dx$$

and $\lambda(x) = (C_{stk}/h)\, x$ is the settling loss rate (bigger droplets, i.e. larger x, fall out faster). Here:

Symbol	Meaning	Units
A_x(x)	Drift velocity in x-space (growth rate)	m^2/s

D_x	Diffusivity in x-space (turbulent broadening)	m^4/s
C_stk	Stokes settling coefficient	1/(m s)
h	Chamber height	m
J_inj	Injection flux of new droplets at small sizes	m^-2 s^-1

An injection flux J_inj is imposed at the small-size boundary (representing new droplets forming), and the flux is set to zero at the largest sizes (nothing grows past the top of the domain).

3. Kohler Theory: Why Small Droplets Behave Differently

The drift $A_x(x)$ depends on how far the ambient supersaturation s is from the droplet's own **equilibrium supersaturation** $s_{eq}(r)$ — the level at which the droplet is neither growing nor shrinking. This equilibrium value is not constant; very small droplets need a higher supersaturation to grow, because of two competing effects:

- **Curvature (Kelvin) effect** — a curved surface evaporates more easily than a flat one, so smaller droplets need more supersaturation to avoid shrinking.
- **Solute (Raoult) effect** — dissolved aerosol material lowers the vapor pressure at the surface, making it *easier* for small droplets to grow.

These combine into the classical Kohler curve:

$$s_{eq}(r) = A_k/r - B_k/r^3$$

where A_k depends on surface tension and B_k depends on the mass of dissolved solute. The curve has a maximum at a **critical radius** r_{crit} and **critical supersaturation** s_{crit} — the activation threshold beyond which a droplet can grow without limit given enough supersaturation. The figure below shows this curve for the dry aerosol radius used throughout this project's example cases (0.065 μm, a typical sub-micron aerosol).

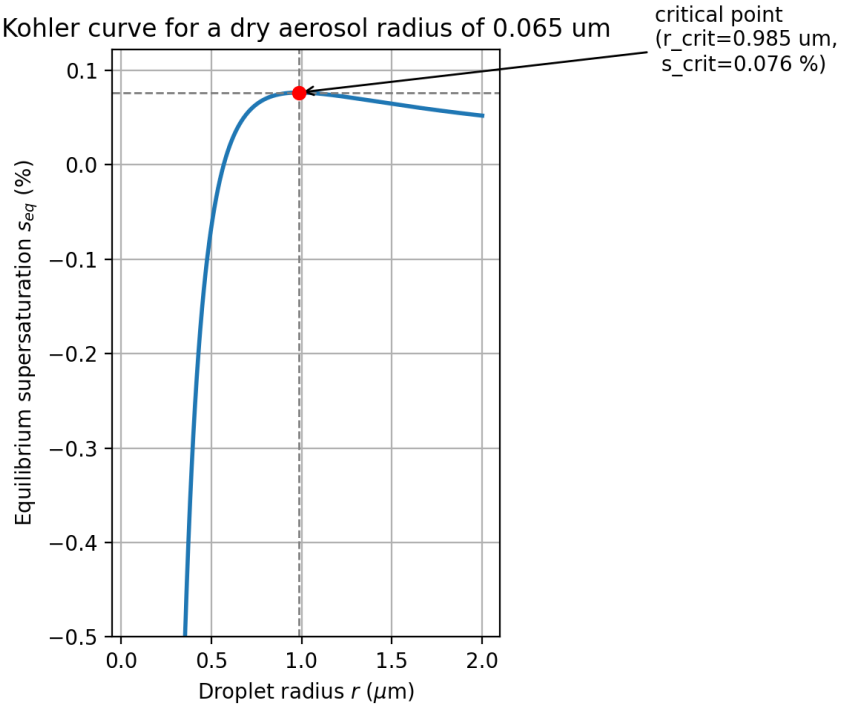


Figure 1. Equilibrium supersaturation vs. droplet radius. The red dot marks the critical (activation) point, computed by `physics.kohler_critical()`.

In the drift formula $A_x(x) = a_{phys}\,(s - s_{eq}(r))$, droplets below their equilibrium curve shrink, and droplets above it grow — exactly the physical behavior this curve encodes.

4. The Numerical Method: Finite Volumes + Scharfetter-Gummel

The domain in x is divided into a large number of cells (8000, in the reference cases). Instead of tracking N(x) at points, the **finite-volume (FV)** method tracks the average number density within each cell, and enforces conservation: whatever flows into a cell across one face, minus what flows out across the other, plus any local settling loss, must balance to zero at steady state.

The tricky part is computing the flux J at each cell face. A naive central-difference approximation becomes unstable and can produce unphysical oscillations (or even negative concentrations) when drift dominates over diffusion — which happens often here, since D_x can be many orders of magnitude smaller than the drift term. The fix used in this code is the **Scharfetter–Gummel (SG) scheme**, originally developed for semiconductor device simulation (where the same drift-diffusion structure appears for electron transport).

At each face, the SG scheme computes a local Peclet number (the ratio of drift-driven to diffusion-driven transport),

$$Pe = A_{x,face}\, dx / D_x$$

and uses the **Bernoulli function** $B(z) = z/(e^z - 1)$ to build exponentially-weighted flux coefficients. This guarantees the resulting solution stays non-negative and remains accurate whether Pe is tiny (diffusion-dominated) or enormous (drift-dominated) — both regimes occur in this project depending on the chosen D_x. In code (`flux.py`), the Bernoulli function is evaluated with three branches to avoid numerical overflow or 0/0 errors: a Taylor expansion very close to z=0, direct evaluation in the middle range, and asymptotic limits for very large |z|.

The resulting system of equations — one balance equation per cell — is **tridiagonal**: each cell's equation only involves its immediate left and right neighbors. This structure is solved directly and efficiently (no iteration required).

5. From Steady State to Time-Dependence

The same spatial discretization extends naturally to a time-dependent version of the equation. Writing the steady system as A N = b, the semi-discrete evolution equation is

$$dN/dt = (b - A\, N) / dx$$

which is advanced using the **backward Euler** method — an implicit scheme that remains stable no matter how large a timestep is chosen (important here, since the settling and diffusion terms can otherwise impose very restrictive stability limits). Each timestep requires solving

$$[I + (dt/dx)\, A]\, N^{n+1} = N^n + (dt/dx)\, b$$

Because the matrix on the left is the same at every timestep (the timestep size dt is fixed), it is factored **once** using sparse LU decomposition and reused for every subsequent solve — making thousands of timesteps computationally cheap.

6. Codebase Tour

The package is organized as a set of small, single-purpose modules under `src/fp_smol/`, each covered by its own test file under `tests/`. The dependency flow is: physics → grid → flux → solver_steady → solver_time.

physics.py

- esat_Pa(T) — saturation vapor pressure over water.
- xi1_func(T, press) — condensation growth-rate coefficient.
- kohler_params(T, r_dry, ...) — computes A_k, B_k.
- kohler_s_eq(r, A_k, B_k) — equilibrium supersaturation curve.
- kohler_critical(A_k, B_k) — critical radius/supersaturation.
- drift_constant / drift_kohler — the two A_x(x) models.
- air_density, stokes_settling_coefficient — supporting physics.

grid.py

- Grid — dataclass holding faces, centers, spacing, radii.
- make_grid(r_min, r_max, Ncells) — builds a uniform grid in x = r^2.

flux.py

- bernoulli(z) — stable evaluation of B(z) = z/(e^z - 1).
- sg_face_coefficients(...) — Scharfetter-Gummel flux coefficients.
- face_average(values) — averages cell-center values onto faces.

solver_steady.py

- settling_loss_rate(xc, C_stk, h) — lambda(x).
- assemble_steady_system(...) — banded tridiagonal A, b.
- assemble_steady_system_sparse(...) — sparse version (used for time-stepping).
- solve_steady(...) — solves A N = b directly, returns N(x).

solver_time.py

- TimeSteppingResult — dataclass of time, N_final, snapshots, error history.
- run_backward_euler(...) — advances N(x,t) forward in time.

7. Installing and Running the Code

This project uses a standard `src/` layout Python package with an isolated virtual environment. From the repository root:

```
# create and activate a virtual environment
python3 -m venv .venv
source .venv/bin/activate      # (Windows: .venv\Scripts\activate)

# install the package in editable mode, plus dependencies
pip install -e .
pip install pytest

# run the full test suite
pytest tests/ -v
```

A minimal example that runs a steady-state solve from scratch:

```
from fp_smol.grid import make_grid
from fp_smol.physics import (xi1_func, kohler_params, drift_kohler,
                             air_density, stokes_settling_coefficient)
from fp_smol.solver_steady import settling_loss_rate, solve_steady

T, p = 288.0, 1000e2          # temperature (K), pressure (Pa)
r_dry = 0.065e-6              # dry aerosol radius (m)
s = 0.0009                    # ambient supersaturation (0.09%)

a_phys = 2 * xi1_func(T, p) / 1e12      # convert to m^2/s
A_k, B_k = kohler_params(T, r_dry)
rho_air = air_density(T, p)
C_stk = stokes_settling_coefficient(rho_air=rho_air)

grid = make_grid(r_min=r_dry/2, r_max=30e-6, Ncells=8000)
Ax_c = drift_kohler(a_phys, s, grid.xc, A_k, B_k)
lam = settling_loss_rate(grid.xc, C_stk, h=1.0)

N = solve_steady(grid, Ax_c, D_x=10/1e24, lam=lam, Jinj=1e8)
r_mean_um = (grid.r_c * N).sum() / N.sum() * 1e6
print(f'Mean radius: {r_mean_um:.3f} um')
```

Common pitfall: the package folder must be named `fp_smol` with an underscore (Python module names cannot contain hyphens), even though the project/distribution name in `pyproject.toml` is `fp-smol` with a hyphen. If `import fp_smol` fails with `ModuleNotFoundError`, check this first.

8. Reproducing the Benchmark Results

The original MATLAB benchmark script produced steady-state radius-PDF plots for a fixed physical setup (s = 0.09%, r_{dry} = 0.065 μm, $a_{phys} \approx 206$ $\mu m^2/s$), varying only the turbulent diffusivity D_x. The Python port (`examples/reproduce_benchmark_plots.py`) reproduces both cases exactly.

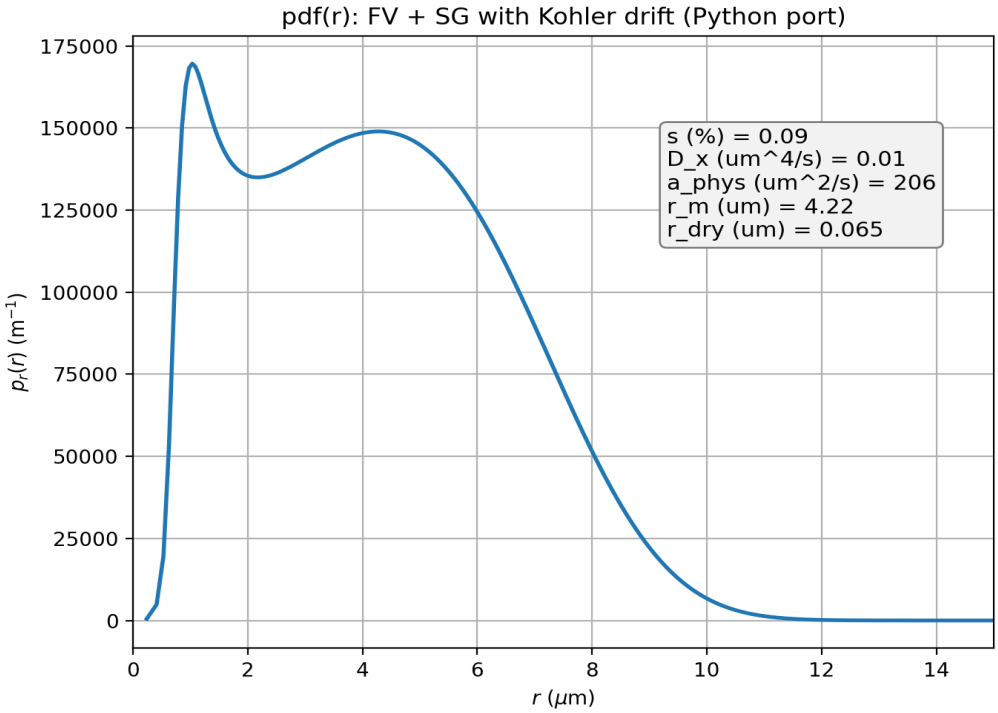


Figure 2. Low diffusivity (D_x = 0.01 μm^4/s). Python computes r_m = 4.222 μm, matching the MATLAB reference value of 4.22 μm.

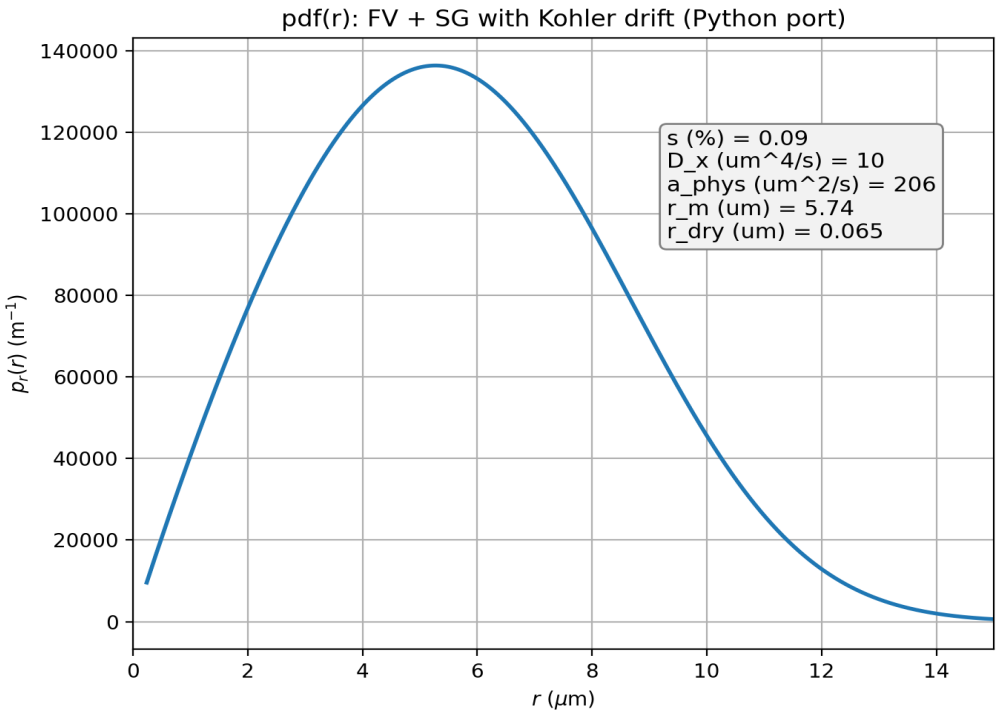


Figure 3. High diffusivity (D_x = 10 μm^4/s). Python computes r_m = 5.736 μm, matching the MATLAB reference value of 5.74 μm.

The physical takeaway matches the theory: more turbulent supersaturation variance (larger D_x) broadens the droplet size distribution and pushes the mean radius upward. Note also the qualitative shape difference — the low-D_x case develops a secondary structure near small radii (droplets that haven't yet diffused away from their near-critical starting point), while the high-D_x case is smoothed into a single broad peak.

9. Validating the Time-Dependent Solver

As a validation exercise, the time-dependent solver is started from an empty chamber (N = 0 everywhere) and integrated forward using the high-diffusivity case above. It should — and does — relax toward the same steady-state solution computed independently by the direct steady solver.

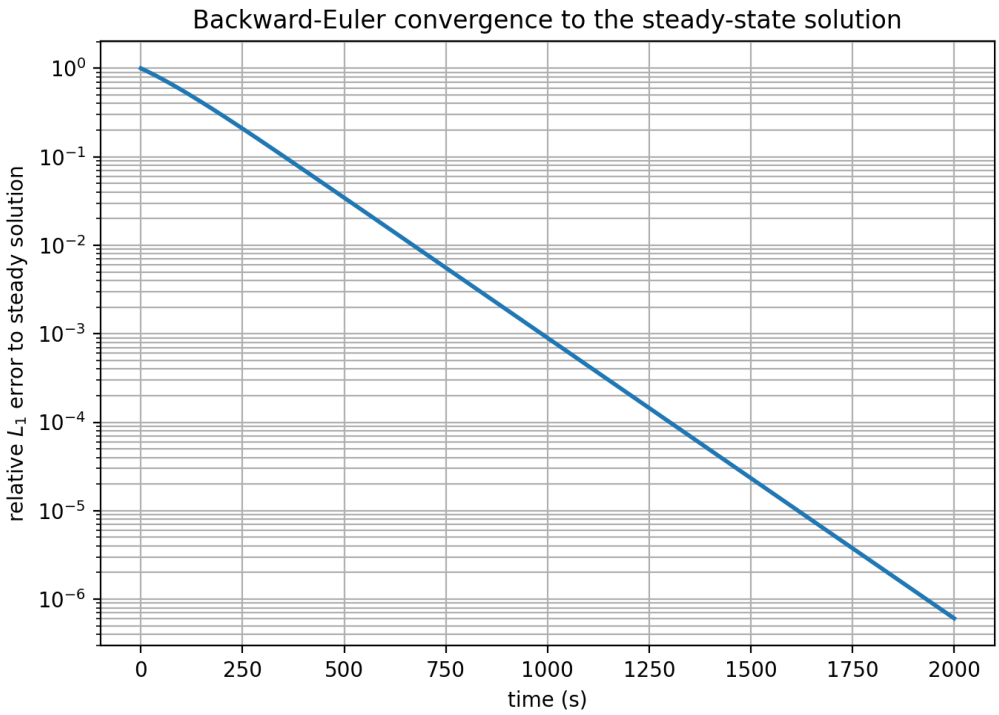


Figure 4. Relative L1 error between the time-dependent solution and the independently-computed steady state, vs. time. The error decays smoothly across many orders of magnitude, with no sign of instability or stagnation.

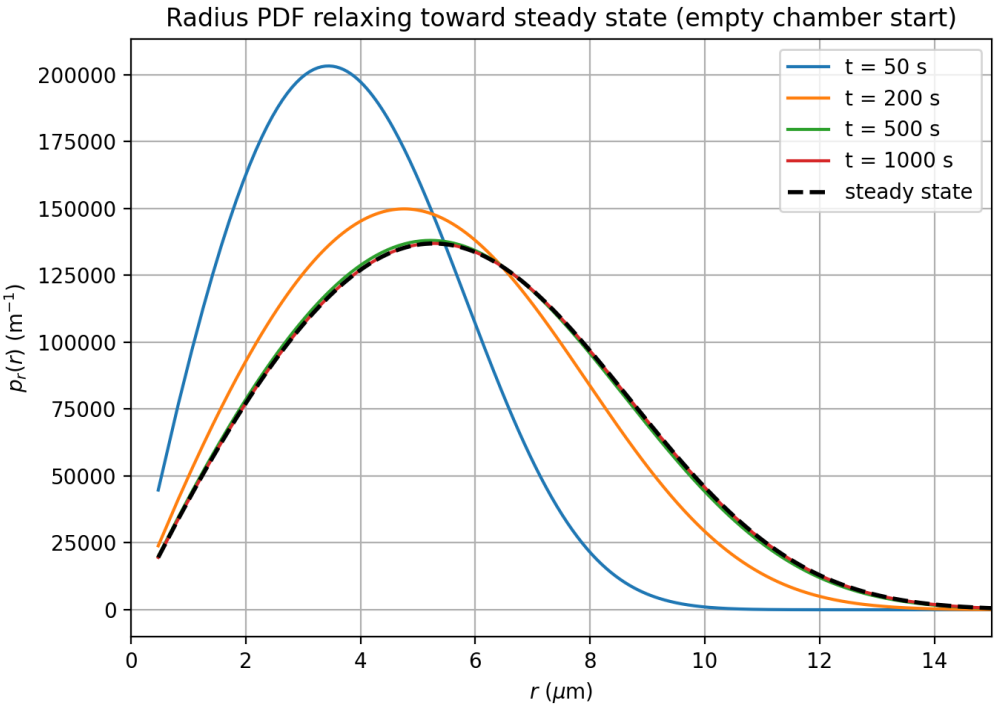


Figure 5. Snapshots of the evolving radius PDF at several times, converging onto the steady-state curve (black dashed). Note the transient overshoot around t=50-200s, where the population briefly exceeds its steady-state shape before settling losses catch up — a genuine physical transient, not a numerical artifact.

10. Extending the Code: What Comes Next

The current codebase covers only **Growth Stage 1** (condensation-driven broadening). The natural next step, flagged in the original project's README as “FP-Smol,” is to add a **Smoluchowski coalescence term** to the governing equation, representing droplets colliding and merging:

df/dt = ... + (production of size x by collisions of smaller droplets) − (loss of size x as it collides with anything else)

This introduces a genuinely nonlinear (quadratic in f) integral term, which will require a different numerical treatment than the linear tridiagonal system used so far — likely an explicit or semi-implicit update for the coalescence integral layered on top of the existing implicit drift-diffusion-settling step. This is exactly the direction the broader ACDC2/Genesis research project is headed, connecting this solver to the Operator-Inference reduced-order modeling and gradient-based parameter estimation workflows described in the project proposal.

Appendix A: Symbol Reference

Symbol	Meaning	Typical value / units
r	Droplet radius	m (plotted in um)
x = r^2	Working variable (radius squared)	m^2
s	Ambient supersaturation	dimensionless (e.g. 0.0009 = 0.09%)
s_eq(r)	Kohler equilibrium supersaturation	dimensionless
A_k	Kohler curvature parameter	m
B_k	Kohler solute parameter	m^3
r_crit, s_crit	Kohler activation point	m, dimensionless
a_phys	Condensation growth-rate coefficient	m^2/s
D_x	Diffusivity in x-space	m^4/s
C_stk	Stokes settling coefficient	1/(m s)
h	Chamber height	m
J_inj	Injection flux at small sizes	m^-2 s^-1
N(x)	Number density in x-space	m^-2 (per unit volume)
p_r(r)	Normalized radius probability density	m^-1
dt, dx	Time step / grid spacing	s, m^2

Appendix B: Troubleshooting Common Issues

ModuleNotFoundError: No module named 'fp_smol'

The package hasn't been installed in editable mode, or the source folder is misnamed. Run `pip install -e .` from the repo root, and confirm the folder is `src/fp_smol` (underscore, not hyphen).

Negative or NaN values in the solution

Usually indicates D_x is too small relative to the grid spacing for the Bernoulli function's small-z branch, or an inconsistent sign convention in a custom drift function. Check that Ax_c has the expected sign (positive = growth) across the domain.

Time-dependent solver disagrees with the steady solver

Confirm both are using assemble_steady_system / assemble_steady_system_sparse built from the *same* grid, drift, diffusivity, and settling arrays — a mismatch here (e.g. different Ncells) will make the two systems physically different problems.

Plots don't match the MATLAB reference values

Double check unit conversions, especially D_x (given in μm^4/s in most scripts but needed in m^4/s internally) and a_phys (from xi1_func, returned in μm^2/s).

End of guide. For the underlying MATLAB reference implementation and original derivations, see the accompanying `matlab_reference/` folder and the project's Fokker-Planck solver description document.